



Git remote

Repozytoria zdalne





Remote (zdalne repozytorium)

to kopia Twojego projektu przechowywana na serwerze — pozwala na pracę zespołową, backup i dostęp z dowolnego komputera. Git od początku był projektowany jako system rozproszony — każdy lokalny klon i tak ma pełną historię, remote to po prostu wspólny punkt synchronizacji.





Repozytorium lokalne

`.git` na Twoim dysku

push →
← pull



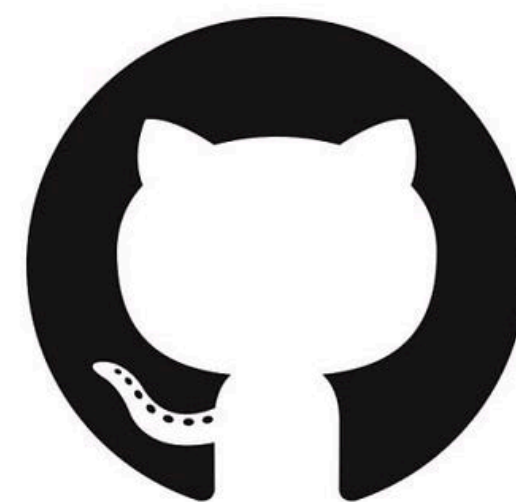
Repozytorium zdalne

np. na GitHub

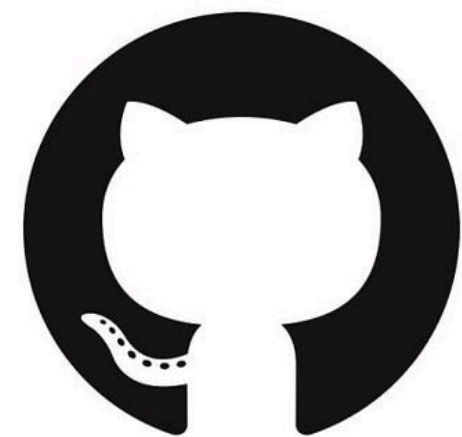


GitHub

Platforma hostingowa zbudowana wokół Gita — nie jedyna, ale najpopularniejsza



GitHub



GitHub

Hosting repozytoriów

Przechowuje Twoje repozytoria w chmurze, dostępne z dowolnego miejsca.

Współpraca

Pull requesty, code review, issues — narzędzia do pracy zespołowej.

Alternatywy

GitLab, Bitbucket, Azure DevOps — ta sama zasada, inny interfejs.

⚠ Przypomnienie

Git ≠ GitHub. Git to narzędzie działające lokalnie; GitHub to jeden z możliwych serwerów zdalnych, na którym trzymasz swoje repozytoria.





git clone — kopiowanie repozytorium

```
$ git clone https://github.com/johnpork/projekt.git
```

```
Cloning into 'projekt'...
```

```
remote: Enumerating objects: 142, done.
```

```
Receiving objects: 100% (142/142), done.
```

*Klonowanie pobiera **całe repozytorium**: pełną historię commitów, wszystkie gałęzie, oraz automatycznie ustawia zdalne repozytorium o nazwie **origin**, wskazujące z powrotem na źródło.*

```
$ cd projekt
```

```
$ git remote -v
```

```
origin https://github.com/johnpork/projekt.git (fetch)
```

```
origin https://github.com/johnpork/projekt.git (push)
```





git clone vs. git init

git init

- Zaczynasz projekt od zera
- Brak podpiętego remote
- Trzeba dodać go ręcznie: `git remote add origin ...`

git clone

- + Dołączasz do istniejącego projektu
- + Remote `origin` ustawiony automatycznie
- + Masz od razu całą historię projektu





git push – wysyłanie zmian

Twój komputer
Repozytorium lokalne
commity gotowe do wysłania

git push →

serwer
Repozytorium zdalne (origin)
GitHub / GitLab

```
$ git push origin main
```

```
Enumerating objects: 5, done.
```

```
Writing objects: 100% (3/3), 312 bytes | 312.00 KiB/s, done.
```

```
To github.com:johnpork/projekt.git
```

```
a1f9c02..7c3e881  main -> main
```





Push wymaga aktualności

```
! [rejected]          main -> main (fetch first)
error: failed to push some refs to 'github.com:johnpork/projekt.git'
hint: Updates were rejected because the remote contains work that
hint: you do not have locally.
```

*Git nie pozwoli wysłać zmian, jeśli ktoś inny wypchnął coś wcześniej, a Ty tego jeszcze nie pobrałeś.
Rozwiązanie: najpierw **git pull**, dopiero potem **git push**.*



git pull – pobieranie zmian

serwer
Repozytorium zdalne (origin)

← git pull

Twój komputer
Repozytorium lokalne

\$ git fetch

Pobiera zmiany z serwera, ale **nie łączy** ich z Twoją bieżącą pracą. Możesz je najpierw obejrzeć.

\$ git pull

git fetch + git merge w jednym kroku. Pobiera zmiany i **od razu scala** je z lokalną gałęzią.





Typowy workflow

1. `git pull origin main` # zacznij od aktualnego stanu
2. # edytujesz pliki...
3. `git add .`
4. `git commit -m "opis zmian"`
5. `git pull origin main` # na wypadek, gdyby ktoś Cię wyprzedził
6. `git push origin main` # wyślij swoje zmiany



Troubleshooting

błąd rejected (fetch first)

Ktoś wypchnął zmiany przed Tobą → zrób `git pull`, potem spróbuj `push` ponownie.

błąd merge conflict

Dwie osoby zmieniły tę samą linię → Git prosi o ręczne rozstrzygnięcie podczas `pull`.

uwaga brak zdalnego

`fatal: No configured push destination` → trzeba dodać origin: `git remote add origin <url>`.

uwaga uwierzytelnianie

GitHub nie akceptuje już haseł przez HTTPS — potrzebny personal access token lub klucz SSH.





:wq

